

Stockdesk ERP and CRM

Full Stack Developer Case Study
Project documentation

Md Raish | 9 September 2026

Stockdesk brings customer follow-ups, inventory and delivery challans into one application. A sales team can track customers and prepare dispatches, while warehouse staff maintain stock and Accounts reviews delivery documents.

The application is live on Render and uses PostgreSQL on Neon. The following pages explain how to access it, run it locally and review the main technical decisions.

Project links

GitHub repository <https://github.com/Mdraish786/stockdesk-erp-crm>

Live frontend <https://stockdesk-erp-crm.onrender.com>

Live backend API base <https://stockdesk-erp-crm.onrender.com/api>

Health check <https://stockdesk-erp-crm.onrender.com/api/health>

Use the frontend link to open the application. The health link returns a JSON response; protected API routes require login.

Document guide

Item	Where to find it
GitHub repository link	Repository link above; deployed commit bda84d3
Live frontend and backend URLs	Public links above; one Express service serves both
Test login credentials for all roles	Evaluator access on page 2
Postman collection or API documentation	docs/Stockdesk.postman_collection.json and docs/API.md
README with setup and deployment instructions	README.md and docs/DEPLOYMENT.md in this package
Short architecture explanation	Page 4 and docs/ARCHITECTURE.md
Known limitations or incomplete parts	Page 6 and docs/VALIDATION.md

Access and local setup

Test accounts

Role	Email	Access
Admin	admin@stockdesk.test	All modules and team management
Sales	sales@stockdesk.test	Customers, follow-ups and challans
Warehouse	warehouse@stockdesk.test	Products and stock movements
Accounts	accounts@stockdesk.test	Read and print delivery challans

Password for all four demo accounts: Stockdesk@2026. Login and dashboard API requests were verified for each role. All roles can read operational records; write access follows the permissions above.

Fast local demonstration on Windows

Install Node.js 22.13 or newer, then run these commands in the VS Code terminal:

```
git clone https://github.com/Mdraish786/stockdesk-erp-crm.git
cd stockdesk-erp-crm
npm ci
npm run demo
```

Open <http://localhost:3000> and sign in with a demo account. Local demo mode uses PGLite and saves records in `.local/stockdesk-pg`. Keep the terminal open while using the app; press Ctrl+C to stop it. If PowerShell blocks `npm.ps1`, use `npm.cmd` instead of `npm`.

PostgreSQL with Docker

Install Docker Desktop and start it. From the project directory, run:

```
npm run setup
docker compose up --build
```

The setup command generates an ignored `.env` file if one does not exist. Docker starts PostgreSQL and the app. Startup creates the schema and demo records, and a named volume preserves database data. The app opens at <http://localhost:3000>.

Checks

```
npm run check:app
npm run check:server
npm run test:api
```

The local run passed both TypeScript checks and all 12 reported tests: 11 business-flow subtests and their parent test. These tests use PGLite. The PostgreSQL workflow result is a separate check in GitHub Actions.

Deployment

Render runs the Express server and serves the built React interface. Neon stores the PostgreSQL database. Frontend and API requests use the same HTTPS origin.

Render setting	Configured value
Runtime and branch	Node; main
Root directory	Repository root; leave the field empty
Build command	npm ci --include=dev && npm run local:build
Start command	npm run local:start
Health check path	/api/health
Plan	Free instance selected during deployment

Environment variables

Variable	Value or source
NODE_VERSION	22
NODE_ENV	production
DB_DRIVER	postgres
DATABASE_URL	Current complete Neon connection string with TLS options
JWT_SECRET	Private randomly generated secret of at least 32 characters
APP_ORIGIN	https://stockdesk-erp-crm.onrender.com
SEED_DEMO	true
DEMO_PASSWORD	Stockdesk@2026 for the seeded evaluation workspace
TRUST_PROXY	1 for the configured Render reverse proxy

Keep DATABASE_URL and JWT_SECRET in Render's environment settings. Render supplies PORT automatically. The hosted app uses postgres; pglite is reserved for local demonstration and tests.

How startup works

At startup, the app checks its configuration, connects to PostgreSQL, applies the schema and seeds missing demo records. Repeated startup preserves existing records and account passwords.

If database authentication fails, update DATABASE_URL with the current Neon connection string and redeploy. APP_ORIGIN must match the public app URL.

The free Render instance can take longer to respond after inactivity. Database records remain in Neon when the web service restarts.

Architecture and business rules

Application structure

The app has three layers. React handles the interface, Express handles requests and business rules, and PostgreSQL stores business records. Protected requests pass through authentication and role checks before a business operation runs.

Component	Responsibility
web/ and components/ui/	Pages, forms, typed API client and reusable UI controls
server/app.ts and auth.ts	REST routes, validation, permissions and JWT sessions
server/challans.ts	Transactional challan and inventory operations
server/db.ts and schema.sql	Database adapter, relational schema and migrations
server/seed.ts	Idempotent initial evaluation accounts and records

Relational model

Each customer can have several follow-ups and challans. A challan contains one or more items, which reference products and retain their original names and prices. Stock movements reference the product, the user who made the change and, where applicable, the related challan.

UUIDs identify records, while a PostgreSQL sequence generates unique challan numbers. Prices use NUMERIC columns. The server calculates totals in SQL and returns monetary values as decimal strings to preserve precision.

Stock changes are transactional

Saving a draft leaves stock unchanged. Confirmation locks the challan and product rows in a consistent order, checks every quantity, deducts stock and records the dispatch. These changes commit together. If any line lacks stock, the entire transaction rolls back.

Repeating confirmation does not deduct stock again. Cancelling a confirmed challan restores its quantities once; cancelling a draft does not affect inventory. Database constraints also prevent negative stock. Saved snapshots preserve the delivery details when the catalogue changes.

Authentication and role enforcement

Passwords are hashed with bcrypt. JWTs expire after two hours and are sent through HttpOnly, SameSite cookies in the browser or bearer headers in API clients. Each protected request checks the active user, role and token version. Logout or deactivation revokes that user's existing sessions.

Zod validates request data, and parameterized SQL keeps user input separate from queries. Permissions are enforced by the API, even when someone sends a request outside the interface.

API documentation and Postman

Canonical base URL: <https://stockdesk-erp-crm.onrender.com/api>. Import docs/Stockdesk.postman_collection.json into Postman. This package sets baseUrl to the live API. Set baseUrl to <http://localhost:3000/api> to use a local demonstration.

The collection includes 39 requests covering login and business flows. It saves bearer tokens after login. A full run creates test records, so use an evaluation workspace.

Main endpoints

Method	Path after /api	Purpose
GET	/health	Public database health
POST	/auth/login	Sign in and obtain bearer token
GET / POST	/auth/me ; /auth/logout	Read current user; revoke sessions
GET	/dashboard	Operational summary
GET / POST	/customers	List or create customers
GET / PUT	/customers/:id	Read or update customer
POST	/customers/:id/followups	Append follow-up note
GET / POST	/products	List or create products
GET / PUT	/products/:id	Read or update catalogue fields
GET / POST	/stock-movements	Read ledger or record stock change
GET / POST	/challans	List or create challans
GET / PUT	/challans/:id	Read or edit a draft
POST	/challans/:id/confirm	Confirm and deduct stock once
POST	/challans/:id/cancel	Cancel and restore stock if needed
GET / POST	/users	Admin team listing or creation
PATCH	/users/:id	Admin activation or deactivation

Authentication example

POST /api/auth/login
Content-Type: application/json

```
{"email": "admin@stockdesk.test", "password": "Stockdesk@2026"}
```

Login returns data.user and data.token. For protected API requests, set Authorization: Bearer <token>. Lists accept page, page_size and search. Responses use data and, for lists, meta; errors use error.code, error.message and optional details. See docs/API.md for complete payloads, filters and status codes.

Validation and known limitations

Verified results

The frontend and backend TypeScript checks passed on Windows. The local API suite reported 12 passed tests and no failures. The local app also loaded the seeded dashboard successfully.

Render successfully deployed commit bda84d3 on 9 September 2026. The public health endpoint returned HTTP 200 with status ok and service stockdesk-api, confirming that the API could query the database.

Live login and dashboard API checks returned HTTP 200 for Admin, Sales, Warehouse and Accounts. These checks confirm API access for each role; they do not cover every browser interaction.

Test coverage and remaining verification

The API suite covers authentication, permissions, CRM validation, stock history, snapshots, rollback, competing requests and token revocation. PGlite runs operations serially, so these results do not prove locking across separate PostgreSQL connections. The GitHub workflow uses PostgreSQL 16; its result has not been confirmed here.

A complete live browser walkthrough, mobile review and stock transition check remain unverified. Docker execution has not been independently checked. The recording is submitted separately through the application form.

Known limitations

- One company workspace; no separate stock balances for multiple warehouses. Stock uses whole units and prices use INR.
- Drafts do not reserve stock. Cancellation returns the entire confirmed quantity; partial returns are not implemented.
- Purchase orders, payment accounting, tax invoices and S3 uploads are not implemented. The printable challan is a delivery document.
- No password-reset email, invitation flow, refresh tokens or per-device session management. Logout invalidates all sessions for that user.
- The deployment install reported 24 dependency vulnerabilities, including one critical. Dependency updates and assessment of runtime exposure remain pending.
- The frontend build reports a large bundle. Future database schema changes need explicit forward migrations.

Supporting files: README.md covers setup; docs/API.md gives complete API payloads; docs/ARCHITECTURE.md explains the design; docs/VALIDATION.md records verification results.